

CMSC 421 Project 1: Informed Search Algorithms for Traveling Salesman Problems (9/8)

1 Introduction

In your first programming assignment, you will use 7 different algorithms to solve the [Traveling Salesman Problem \(TSP\)](#). This is a challenging assignment that asks you to implement the algorithms and understand them, so that you can discuss the trade-offs, usability, and hyperparameters of each algorithm. To do this, you are encouraged to discuss the project with your classmates or TAs and to use whichever programming language you are comfortable in.

Be mindful that this is an open-ended project without correct answers. You are encouraged to experiment with the algorithms and infer your own results. A* is the only algorithm that is theoretically guaranteed to find the correct solution, however, even a perfect implementation may struggle problems with more than 10 cities.

1.1 Required Submission

You will submit 3 items when you have finished the project:

1. A PDF report of your findings: The main element of the project is a written report that includes your findings, including your answers to the questions in each section. Your report should be well written and include clear, nice-looking plots, like those shown in Figure 1. The figures can be made in any software of your choice, including Excel (by saving the results as a CSV and loading it), [matplotlib](#), [matlab](#), or [plotly](#).
2. Code file(s): The other essential part of your project is the written code. You should submit all of the code you have written either in a single file or in separate files for each part of the project. The code must be runnable by the TAs and be your own work, but can be in any language you choose (so please include compile/environment setup instructions). **Each code file should be able to be run on a file provided as a command line argument. This can be done using `sys.argv` in Python or `String args[]` in Java. The file format is discussed in Section 1.2. When you run the file from the command line, you should either print out the runtime, CPU time, and score or save it to a file.** Please limit the functions you use to the standard library, [numpy](#), plotting libraries, the [scipy](#) function discussed below, and [AIMA](#).
3. Screen recording: Finally, please submit a screen recording of you running each part of the project. This is required to help the TAs grade your assignment. You should be able to see each section run based on an input file, with a resulting terminal output or file output. On Windows, press the Windows Key and search for **Snipping Tools** to record. On Mac, use CMD-Shift-5 to record.

1.2 Input Files

The traveling salesman problem can be described by an adjacency matrix, which must be either symmetric or triangular, depending on how you implement your algorithms. Therefore, the file format to save the TSP problem is simply a matrix, with

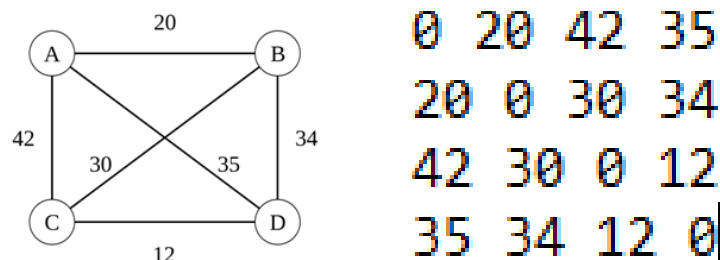


Figure 1: An example TSP problem and the corresponding adjacency matrix.

rows delimited by new lines and columns delimited by spaces. This file format can be generated by `np.savetxt(fname, mat)` and loaded by `mat = np.loadtxt(fname)`.

10 random adjacency matrices for sizes 5, 10...25, 30 are also provided (along other sizes in case your algorithm cannot run past 10 cities or you want to collect more data), which can be used in the experiments. Additionally, adjacency matrices with cities evenly distributed on the edge of a unit-side-length n -gon will be provided. These can be used to test your algorithm since you know that the shortest path is equal to n (the path that traces the parameter of the n -gon is the shortest path), **but should not be used for the experiments**.

2 Part I: Nearest Neighbor, Nearest Neighbor 2-Opt, Repeated Randomness

Nearest Neighbor

In this section, you will implement the three easiest algorithms. These are greedy algorithms that should run quickly for well over 50 cities, but, as you will see below, will generate less-optimal solutions than A*

2.1 Algorithm Setup

You must submit (at least) three functions, described here:

- Nearest neighbor: This algorithm is by far the most simple. It should take in a single argument, the adjacency matrix, and, starting with an arbitrary node, choose the next closest unvisited node until all are visited, then returning to the initial node.
- Nearest neighbor with 2-Opt: Starting with your nearest neighbor algorithm, add the [2-opt algorithm](#) to refine your solution. This algorithm repeatedly tests all possible swaps of adjacent cities to see if the swap improves the score. When a swap that improves the solution is found, perform the swap and restart by looking for new beneficial swaps from the beginning of the route. When all pairs of cities do not result in a beneficial swap, return the route.
- Repeated random nearest neighbor (RRNN): Write a similar algorithm to nearest neighbor with 2-opt with two differences: (1) instead of automatically choosing the nearest neighbor, randomly choose the next city from the k closest cities and (2) repeat this process *num_repeats* times, choosing the best solution. Therefore, add the hyperparameters *k* and *num_repeats* to your function.

2.2 Experiment Setup

To test your algorithms, generate or use the 10 random adjacency matrices of sizes 5, 10...25, 30. First, use these matrices to find optimal hyperparameters for your RRNN algorithm. To do this, run the algorithm on the matrices and create two plots, one with different values of *k* on the *X* axis and the median score on the *Y* axis, and another with different values of *num_repeats* on the *X* axis and the median score on the *Y* axis. Make sure to keep one parameter constant as you test the other parameter. Use the best parameters when you compare the algorithms below.

Then, compare the three algorithms to each other. To do this, run each algorithm on each of the generated matrices, finding the median wall time required, median CPU time required, and median cost to traverse the cities. You can use [process_time_ns](#) and [time_ns](#) to track the time in Python. The score should be the sum of the distances from the adjacency matrices of the route taken. If the CPU time is zero, run the algorithm multiple times for each matrix before stopping the time and that value divided by the times ran.

Use these results to create 3 plots, one for total time, one for CPU time, and one for the solutions' scores. For the first plot, put each algorithms' runtime on the *Y* axis and the number of cities on the *X* axis. Each tick on the *X* axis should show size of the problem from 5, 10...25, 30. On the second plot, repeat the same process but put the algorithm's CPU time on the *Y* axis. Finally, for the third plot, put each algorithms' score on the *Y* axis. Remember to label and title your figure, using color or shapes to distinguish between the algorithms. Please reference Figure 2 or Figure 3 for examples of how to make an appealing figure.

2.3 Questions to Consider in Your Report

Please answer these question in the report, alongside your figures

- Do the algorithms here find optimal paths to solve the problem? Why or why not?

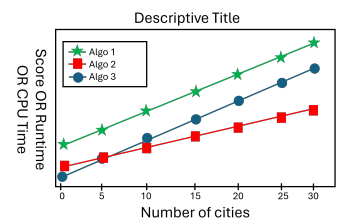


Figure 2: The "Platonic Ideal" of a plot for this assignment. A good plot (when applicable) should show the number of cities on the *X* axis, with the score, runtime or CPU time on the *Y* axis. The algorithms should be plotted in clearly distinct ways, with a descriptive legend and title.

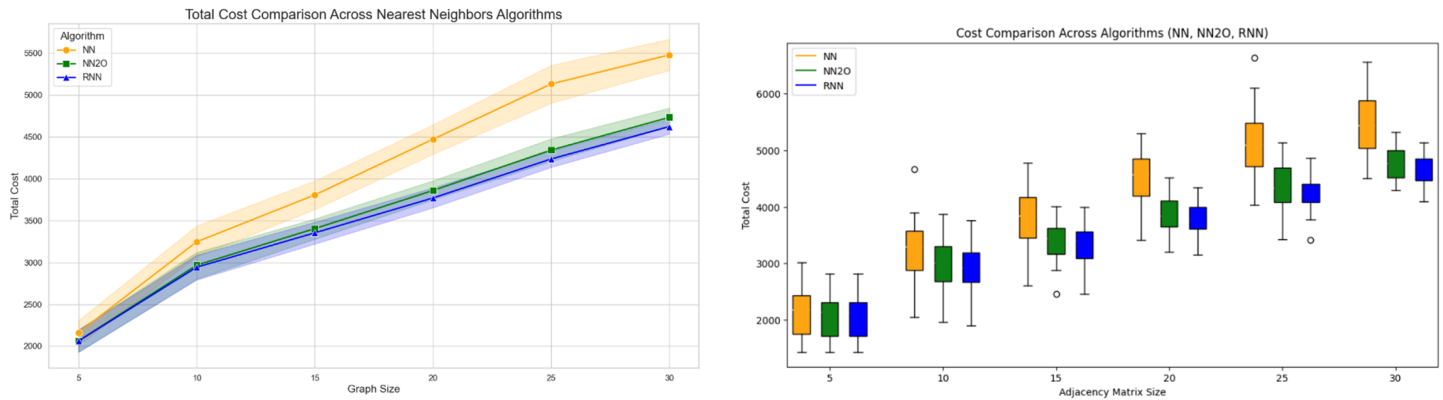


Figure 3: Examples of top-tier figures for your inspiration. When creating your figures, remember to include axis labels, a descriptive title and a legend. Use colors or shapes to differentiate different algorithms.

- What values did you choose for k ? What are the tradeoffs for lower or higher values?
- What values did you choose for `num_repeats`? What are the tradeoffs for lower or higher values?
- Which algorithm found a solution the quickest? Was this at the cost of solution accuracy?
- Using n , k , and `num_repeats`, what are the time complexities of the three algorithms? Do the time complexities effect the figures you generated?

3 Part II: A* with MST Heuristic

In this section, you will implement the most challenging algorithm, A*, using the MST heuristic. This algorithm will run much slower than the other algorithms but is guaranteed to find an optimal solution. Problems with numbers of cities $1 \dots 10$ are included with the other problems if necessary.

3.1 Algorithm Setup

Submit (at least) two functions, described here:

- Minimum-spanning tree heuristic: Using an algorithm like [Prim's algorithm](#) or [Kruskal's algorithm](#), implement a function that calculates a heuristic cost for each city by summing the edges of a [minimum spanning tree](#). The function can take any arguments you want, but, given a city and a list of already-visited cities, you should return the total cost of the minimum-spanning tree starting from the given city and connecting to every unvisited city. **Pseudocode for implementing Prim's Algorithm is available online, or you can use the [scipy function](#).**
- A*: Implement the A* function, where $g(x)$ is the cost to get to each city and $h(x)$ is the minimum-spanning tree from the previous function. The function should input a matrix and return a complete route, beginning and ending at the same city. This is probably the hardest part of the project, but there are many resources that can assist you online, including [AIMA](#). Keep in mind that part of the challenge of the project is coming up with your own implementation, and try to optimize your algorithm, as it should run for matrices of 10 – 15 cities. When writing the algorithm, consider that the current 'state' each iteration is the partial 'tour', and your success function tries to add another city to the 'tour' based on the cost to reach that city and the heuristic cost of completing the 'tour' from that city.

3.2 Experiment Setup

To test your algorithms, run A* on the same matrices from the previous part, stopping when it becomes impossible to run A* on that size of problem, which will occur because the time required by the A* algorithm will quickly increase as the number of cities grows. There are additional matrices with $1 \dots 10$ cities in them if that works better on your algorithm, but you will have to run the same graphs on both this algorithm and the previous ones to compare them. Then, recreate the same 3 plots comparing nearest neighbor, nearest neighbor with 2-opt, and RRNN. To compare the algorithms to A*, divide each value on the Y axis by the corresponding value for A*. For example, for one figure, plot the CPU time for each algorithm divided by the CPU time for A* on the Y axis and the size of the matrix on the X axis. If the A* algorithm took 1 second for graphs of $n = 10$, 2 for $n = 20$, and 3 for $n = 30$, and RRNN took 0.5 seconds for $n = 10$, 1.5 for $n = 20$, and 3 for $n = 30$, you would plot 0.5, 0.75, 1.0 for RRNN. **On the plots for the algorithms' runtimes and the algorithms' CPU times,**

please plot the median nodes expanded by A* on a second Y axis. A node is expanded when the heuristic is calculated on it and the number of nodes expanded should increase quicker than the number of cities. Again, make sure your figures are complete by adding a title and labels.

3.3 Questions to Consider in Your Report

Answer these questions by providing the figures in your report.

- How are solutions represented by your version of A*? How did your choice of representation affect your program (consider this from a software engineer's perspective)?
- What number of cities were you unable to run? What part of the algorithm, do you think, increased the time the most? How could you improve the algorithm?
- Discuss how the number of nodes expanded increases as the number of cities increases. Discuss how this relates to the previous question.
- How does A* compare to the other algorithms, in both time and solution cost? Is A* a practical algorithm for real world use?

4 Part III: Hill Climbing, Simulated Annealing, Genetic Algorithms

In this section, you will implement Hill Climbing, Simulated Annealing, and a Genetic Algorithm. Hill climbing, simulated annealing, and genetic algorithms are surprisingly strong algorithms despite their simplicity.

4.1 Algorithm Setup

You will write (at least) three functions:

- Hill climbing: Create a randomly-restarting [hill climbing](#) function that solves the problem. For *num_restarts*, start by generating a random solution to the problem. Then, generate random "neighboring" solutions by choosing two nodes to swap, and, if the solution improves the results, replace the source solution with it. After *num_restarts*, return the best solution found. To do this, your function should take in both the adjacency matrix and `num_repeats`.
- Simulated annealing: Your [simulated annealing](#) implementation should be similar to the implementation of hill climbing. Again, starting with a random solution, find a "neighboring" solution. Then, choose to replace the original solution with the new solution if it is less costly or with the probability

$$e^{\frac{score' - score}{t}}$$

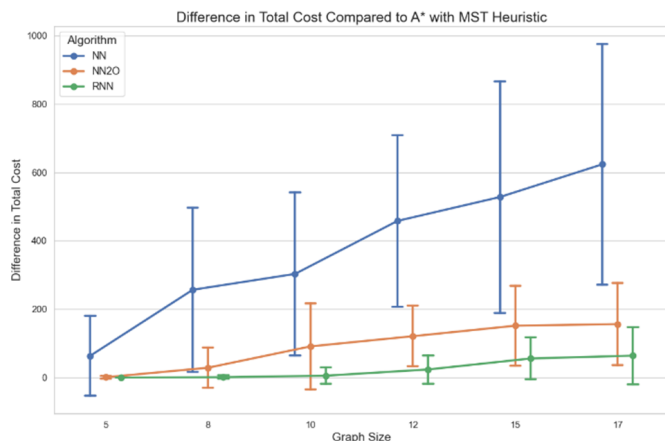
where *t* is the temperature. Your algorithm should take in the adjacency matrix, `alpha` (how much the temperature decreases when a route is accepted), `initial_temperature`, and `max_iterations`.

- Genetic algorithm: the genetic algorithm starts with a population of randomly generated solutions, and, each iteration, pairs of "parents" are chosen to create "child" solutions. This crossover can be accomplished using any of the techniques in section 4.3 of [Larrañaga et al.](#), and the child should be mutated (cities randomly swapped) with *mutation_chance* probability. After all children are created, the combined list of children and parents should be sorted by score and the lower solutions should be discarded. This last step is called "elitism" because the most elite members of the population are kept between generations. The algorithm should take in the adjacency matrix, `mutation_chance`, `population_size`, and `num_generations`, and it should return the solution with the lowest cost.

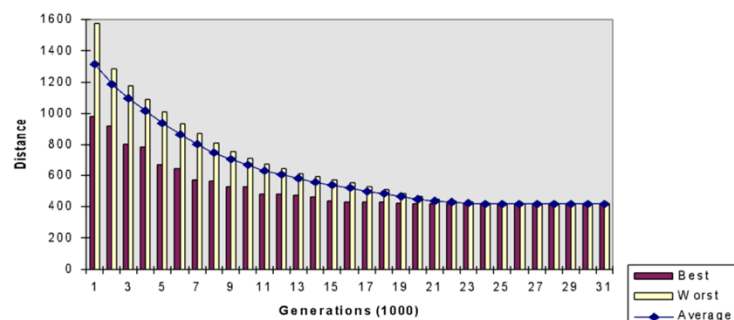
4.2 Experiment Setup

Create **at least one figure for each algorithm**, highlighting how **at least one** hyperparameter change the algorithms' solutions' score. Additionally, create **at least one figure for each algorithm** figure that shows how the score of each algorithm improves over each generation or iteration. Do this by calculating the best score that the algorithm has found on each iteration and plotting it on the Y axis. Then, plot the generation or iteration on the X. The score should be a median taken on all of the figures of a set size, and each matrix should be repeated with the best score taken (i.e. what happens with *num_restarts*).

Then, like the figures previously created for Part II, create three figures that highlight the difference in performance between your hill climbing, simulated annealing, and genetic algorithms. These figures should show the runtime, CPU time, and score divided by A*'s performance on the previous matrices. For example, for one figure, plot the CPU time for each algorithm divided by the CPU time for A* on the Y axis and the size of the matrix on the X axis. Again, make sure your figures are complete by adding a title and labels.



(a) This figure is an example of a plot that shows how the nearest neighbor-based algorithms or the Section 4 algorithms compared with A*, which can be accomplished by dividing the score of each algorithm by A*'s respective score. Your plot showing comparing your algorithms to A* stop be similar.



(b) This figure is an example of a plot that shows how the cost of the genetic algorithm's best solution decreases as the number of generations increases. Your plot showing your genetic algorithm in Section 4 should look similar.

4.3 Questions to Consider in Your Report

Please answer these question in the report, alongside your figures

- How are solutions to the traveling salesman problems represented by these algorithms? What does it mean to be a neighbor to one of these algorithm's solutions?
- What hyperparameters were the best for each algorithm? Discuss the tradeoffs between different values for at least one hyperparameter per algorithm
- Compare the performance of these algorithms with the performance of the nearest neighbor algorithms. What makes these algorithms better or worse choices than the other algorithms?
- How do these algorithms compare with A*? When might you want to use A* over one of these algorithms? When might you use these algorithms over A*?
- How do these algorithms harness randomness to improve on deterministic algorithms like nearest neighbor or A*?
- Discuss four different real-world applications of the traveling salesman problem. Which algorithm would you chose (from all of the algorithms discussed) for each scenario? Try to choose setups and scenarios where different elements (time, cost, etc) are more relevant.

5 Part IV: Extra Credit

To participate in the extra-credit, generate a path for the provided `extra_credit.txt`, which is a really large matrix. Then, submit the path **and** add a paragraph to your report on the modifications made/algorithms run to find this solution. We will calculate the points you receive as

$$1/e^x * 10$$

where x is the percentile of your solution compared to the other solutions. Even the worst solution will receive points under this model, so it makes sense to submit something even if you cannot run your advanced algorithms.

6 Figure Checklist

1. Two plots showing the effects of different hyperparameters on RRNN
2. Three figures showing the CPU time, runtime, and cost for NN, NN-2opt, and RRNN for various numbers of cities
3. Three figures, the same as above, but with the CPU time, runtime, and cost divided by A*'s respective statistics
4. One figure showing how the number of nodes expanded increases as the number of cities increases

5. Three figures highlighting how different hyperparameters change hill climbing, simulated annealing, and the genetic algorithms
6. Three figures showing how the cost of the genetic algorithm's, hill-climbing's, and simulated annealing's solutions decreases over each iteration
7. Three figures, the same as above, but with the CPU time, runtime, and cost of **hill climbing, simulated annealing, and the genetic algorithm** divided by A*'s respective statistics

All Figures should show the median or mean over multiple trials for reproducibility. Additionally, please make sure that you answer all of the questions provided, and discuss how you wrote the algorithms and ran the experiments.